

12⁺ Operator Overloading

Conversion Operators

Hu Sikang
skhu@163.com

Conversion Operators

Implicit conversion from building type to class type

```
#include <iostream>
using namespace std;
class complex {
public:
    complex(double x = 0, double y = 0) {
        re = x;    im = y;
    }
```

```
    friend complex operator +(const complex& a, const complex& b);
```

```
private:
```

```
    double re, im;
```

```
};
```

```
complex operator +(const complex& a,
                  const complex& b)
```

```
{
```

```
    double m = a.re + b.re;
```

```
    double n = a.im + b.im;
```

```
    return complex(m, n);
```

```
}
```

operator+ (x, complex(3));

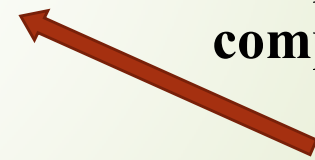
```
int main( ) {
    complex x(10, 20);
    complex z;
```

```
    z = x + 3; //ok
```

```
    z = 3 + x; //ok
```

```
    return 0;
```

```
}
```



Conversion Operators

Another implicit conversion from class type to other types

```
complex obj(1, 2);
```

```
int x;
```

```
x = obj;    // conversion operators
```

- [1] an implicit conversion from a user-defined type to a building type, or
- [2] a conversion from a new class to a previously defined class.

Conversion Operators

```
#include <iostream>
using namespace std;
class Rational {
public:
    Rational(double x = 0, double y = 1)
    {
        Numerator = x;
        Denominator = y;
    }
    operator double() {
        return Numerator / Denominator;
    }
private:
    double Numerator, Denominator;
};
```

```
int main( )
{
    Rational r(100, 200);
    double d;

    // conversion operator
    d = r;

    cout << d << endl;
    return 0;
}
```

Conversion Operators

```
#include <iostream>
using namespace std;
class A { };
class B {
public:
    operator A() {
        cout << " Convert B to A!" << endl;

        // Return an anonymous object of class A
        return A( );
    }
};
```

How should we do if the statement is "*b = a*"?

```
int main( ) {
    B b;
    A a;
    a = b; //convert
    return 0;
}
```

Exercise (1)

```
#include <iostream>
using namespace std;
class Rational {
public:
    Rational(double x = 0, double y = 1)
    {
        Numerator = x;
        Denominator = y;
    }
    operator double( ) {
        return Numerator / Denominator;
    }
private:
    double Numerator, Denominator;
};
```

```
int main( )
{
    Rational r1(100, 200), r2;
    double d = r1;           // (1)
    r2 = r1 + d;             // (2)
    cout << r2 << endl;    // (3)

    return 0;
}
```

Question:

Is the statement (3) right?

Exercise (2)

```
#include <iostream>
using namespace std;
class Rational {
public:
    Rational(double x = 0, double y = 1)
    {
        Numerator = x;
        Denominator = y;
    }
    operator double()
    {
        return Numerator / Denominator;
    }
};
```

Question:

Is the statement (2) right?

```
friend Rational operator+ (Rational& left, Rational& right)
{
    return Rational(left.Denominator + right.Denominator,
                    left.Numerator + right.Numerator);
}
private:
    double Numerator, Denominator;
};
int main() {
    Rational r1(100, 200), r2;
    double d = r1;           // (1)
    r2 = r1 + d;             // (2)
    cout << r2 << endl;     // (3)

    return 0;
}
```

Exercise (2)

```
#include <iostream>
using namespace std;
class Rational {
public:
    Rational(double x = 0, double y = 1)
    {
        Numerator = x;
        Denominator = y;
    }
    operator double()
    {
        return Numerator / Denominator;
    }
};
```

```
double ToDouble()
{ return Numerator / Denominator; }
```

```
friend Rational operator+ (Rational& left, Rational& right)
{
    return Rational(left.Denominator + right.Denominator,
        left.Numerator + right.Numerator);
}
private:
    double Numerator, Denominator;
};
int main( ) {
    Rational r1(100, 200), r2;
    double d = r1;           // (1)
    r2 = r1 + d;              // (2)
    cout << r2 << endl;     // (3): What should we do next?

    return 0;
}
```

It's recommended not to use automatic conversion as much as possible.

Exercise (3)

```
#include <iostream>
using namespace std;
class vector {
public:
    vector(int s = 0) { v = new int[s]; size = s; }
    ~vector() { if (v != nullptr) delete[] v; }
    int& operator [ ](int i) { return v[i]; }
private:
    int *v;
    int size;    // number of elements
};
```

```
int main( )
{
    vector vec;
    vec[2] = 12;
    cout << vec[2] << endl;
    return 0;
}
```

If the code is written as follow, how to modify *Vector*:

```
if (vec) // Is the "vec" empty?
    vec[2] = 12;
else
    cout << "No element in vec." << endl;
```